Substitution and the Lambda Calculus

Concepts of Programming Languages

CAS CS 320

Practice Problem

```
<prog> ::= { <stmt> ; }
<stmt> ::= rot90 | refX | refY
```

$$\frac{}{(s,\ \texttt{rot90;}\ P) \longrightarrow (s \text{ rotated 90 deg. clockwise},\ P)}$$

$$\frac{}{(s,\ \texttt{refX;}\ P) \longrightarrow (s \text{ reflected across x-axis},\ P)}$$

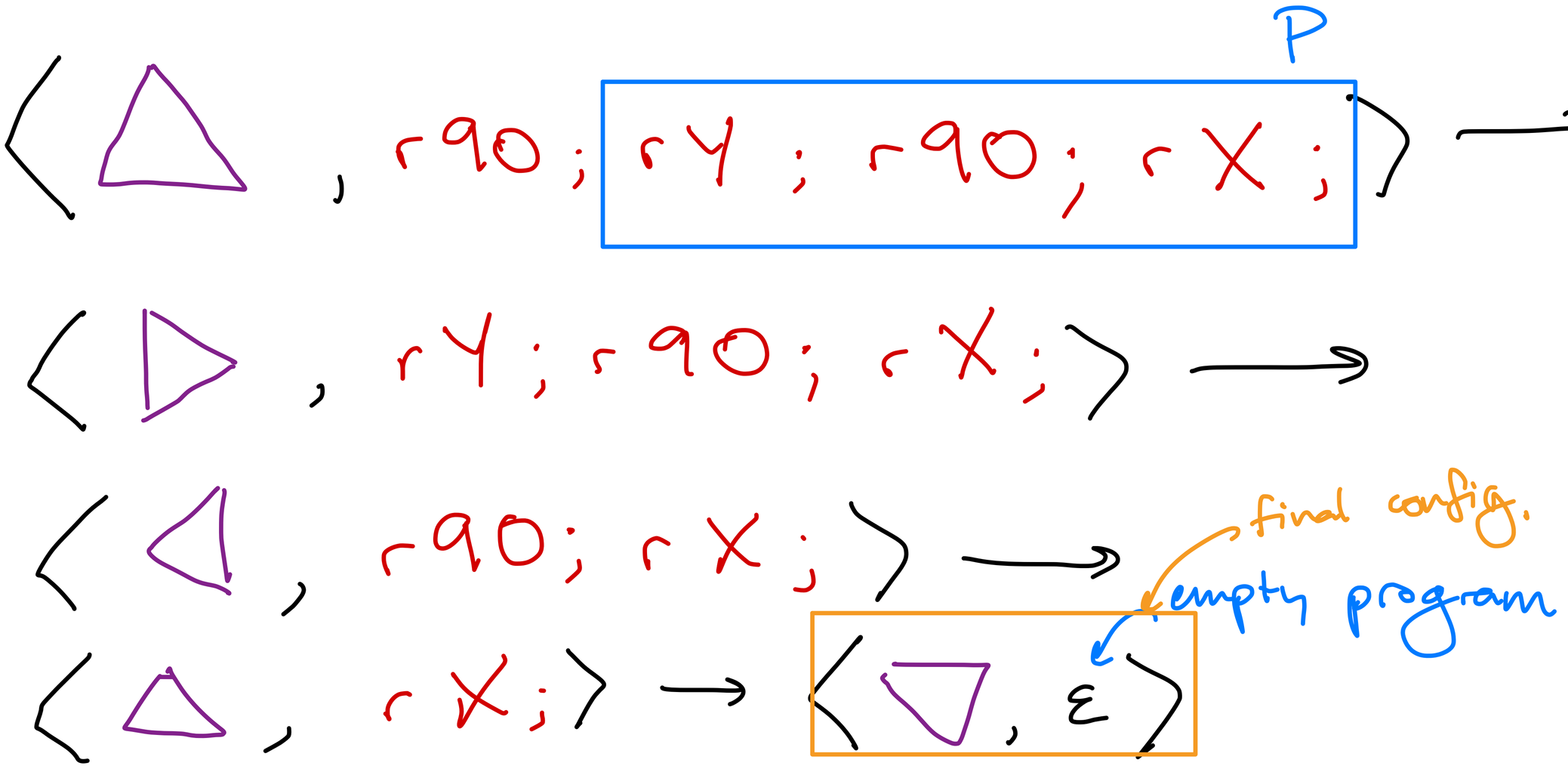
$$\frac{}{(s,\ \texttt{refY;}\ P) \longrightarrow (s \text{ reflected across y-axis},\ P)}$$

What does (△, rot90; refY; rot90; refX;) *evaluate to?*
Provide a trace

Answer

```
<prog> ::= { <stmt> ; }
<stmt> ::= rot90 | refX | refY
```

(△, rot90; refY; rot90; refX;)

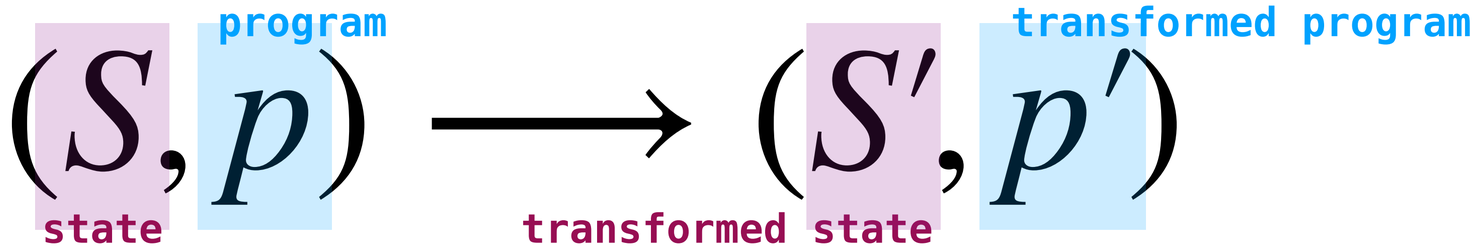


Outline

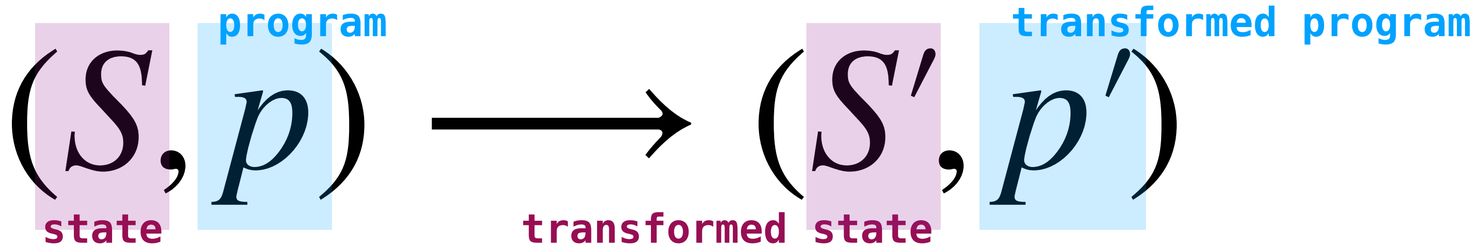
- Look formally at the **lambda calculus** and its semantics
- Discuss **substitution** and the pitfalls to avoid
- Demo an **implementation** of the lambda calculus

Recap

Recall: High-Level

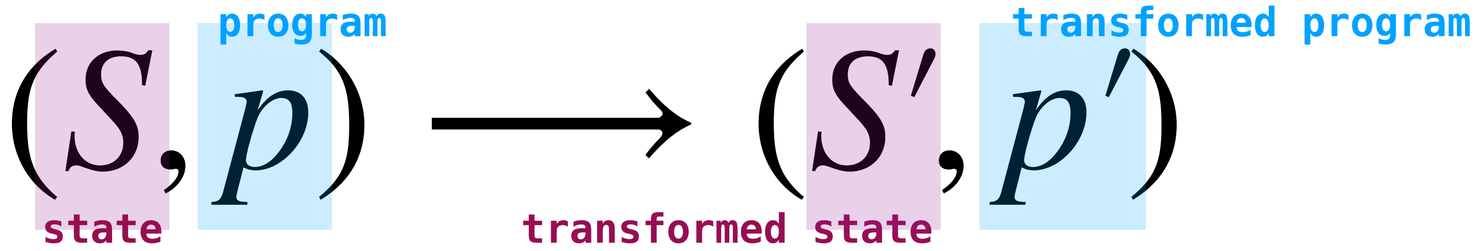


Recall: High-Level



When we define the small-step semantics of PL, we need to define three things:

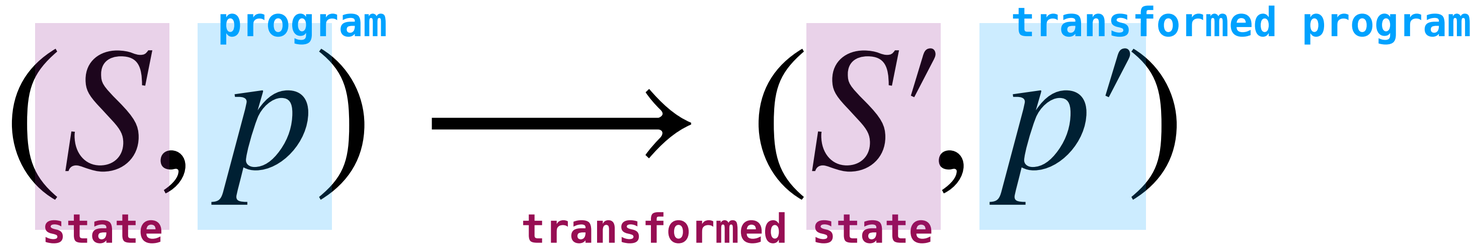
Recall: High-Level



When we define the small-step semantics of PL, we need to define three things:

» What kind of **state** are we manipulating?

Recall: High-Level

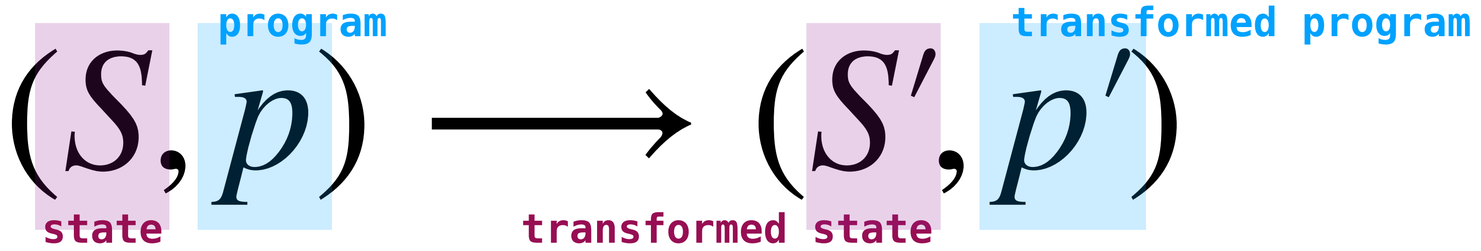


When we define the small-step semantics of PL, we need to define three things:

» What kind of **state** are we manipulating?

» What **rules** describe how to transform configurations?

Recall: High-Level



When we define the small-step semantics of PL, we need to define three things:

» What kind of **state** are we manipulating?

» What **rules** describe how to transform configurations?

» What are the **values** of our PL?

Recall: Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

Recall: Example

State: ∅

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

Recall: Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

State: $\varnothing$

Rules:

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1'\ e_2)}\ \texttt{add-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{add}\ n\ e_2) \longrightarrow (\texttt{add}\ n\ e_2')}\ \texttt{add-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{add}\ n_1\ n_2) \longrightarrow n_1 + n_2}\ \texttt{add-ok}$$

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{sub}\ e_1\ e_2) \longrightarrow (\texttt{sub}\ e_1'\ e_2)}\ \texttt{sub-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{sub}\ n\ e_2) \longrightarrow (\texttt{sub}\ n\ e_2')}\ \texttt{sub-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{sub}\ n_1\ n_2) \longrightarrow n_1 - n_2}\ \texttt{sub-ok}$$

Recall: Example

```
<expr> ::= ( <op> <expr> <expr> )
         | <bool> | <int>
<op>   ::= add | sub | eq
<bool> ::= true | false
<int>  ::= ...
```

State: $\varnothing$

Rules:

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{add}\ e_1\ e_2) \longrightarrow (\texttt{add}\ e_1'\ e_2)}\ \texttt{add-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{add}\ n\ e_2) \longrightarrow (\texttt{add}\ n\ e_2')}\ \texttt{add-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{add}\ n_1\ n_2) \longrightarrow n_1 + n_2}\ \texttt{add-ok}$$

$$\frac{e_1 \longrightarrow e_1'}{(\texttt{sub}\ e_1\ e_2) \longrightarrow (\texttt{sub}\ e_1'\ e_2)}\ \texttt{sub-left}$$

$$\frac{n\ \texttt{is a number} \qquad e_2 \longrightarrow e_2'}{(\texttt{sub}\ n\ e_2) \longrightarrow (\texttt{sub}\ n\ e_2')}\ \texttt{sub-right}$$

$$\frac{n_1\ \texttt{is a number} \qquad n_2\ \texttt{is a number}}{(\texttt{sub}\ n_1\ n_2) \longrightarrow n_1 - n_2}\ \texttt{sub-ok}$$

Values: <int> | true | false

The Lambda Calculus

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

The **lambda calculus** is the *simplest functional programming language*. It has:

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

The **lambda calculus** is the *simplest functional programming language*. It has:

» variables

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

The **lambda calculus** is the *simplest functional programming language*. It has:

- variables
- anonymous functions

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

The **lambda calculus** is the *simplest functional programming language*. It has:

- variables
- anonymous functions
- function application

High-Level View

```
(fun x -> x x)(fun x -> x x)
```

lambda term called Ω

The **lambda calculus** is the *simplest functional programming language*. It has:

- variables
- anonymous functions
- function application

It's **untyped**, so *anything* can be applied to *anything*

demo

(OCaml and Python)

History



History

The lambda calculus was introduced by **Alonzo Church** in the 1930s



History

The lambda calculus was introduced by **Alonzo Church** in the 1930s

Church was trying to give a *foundation of mathematics* (did not succeed) and extracted from that work the lambda calculus



History

The lambda calculus was introduced by **Alonzo Church** in the 1930s

Church was trying to give a *foundation of mathematics* (did not succeed) and extracted from that work the lambda calculus

The lambda calculus **as powerful** as every model of computation (Turing Machines, Register Machines, etc.)



Syntax

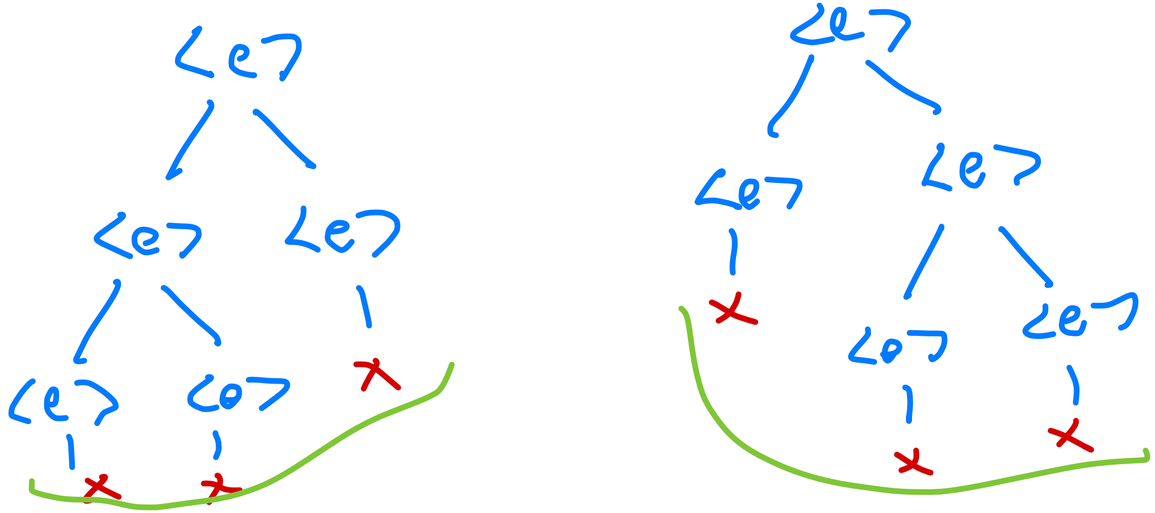
Concrete Syntax

```
<expr> ::= fun <var> -> <expr>
         | <var>
         | <expr> <expr>
         | ( <expr> )
<var>  ::= ...
```

Concrete Syntax

```
<expr> ::= fun <var> -> <expr>
         | <var>
         | <expr> <expr>
         | ( <expr> )
<var>  ::= ... x
```

This presentation is ambiguous (why?)



Concrete Syntax

```
<expr> ::= fun <var> -> <expr>
         | <var>
         | <expr> <expr>
         | ( <expr> )
<var>  ::= ...
```

This presentation is ambiguous (why?)

We assume application is **left-associative** and has higher precedence than the anonymous function syntax

Concrete Syntax (Unambiguous)

```
<expr>  ::= fun <var> -> <expr>
          | <expr2> { <expr2> }
<expr2> ::= <var> | ( <expr> )
<var>   ::= ...
```

Abstract Syntax (Mathematical)

anon fun | app | var

$$e ::= \lambda x.e \mid e\ e \mid x$$

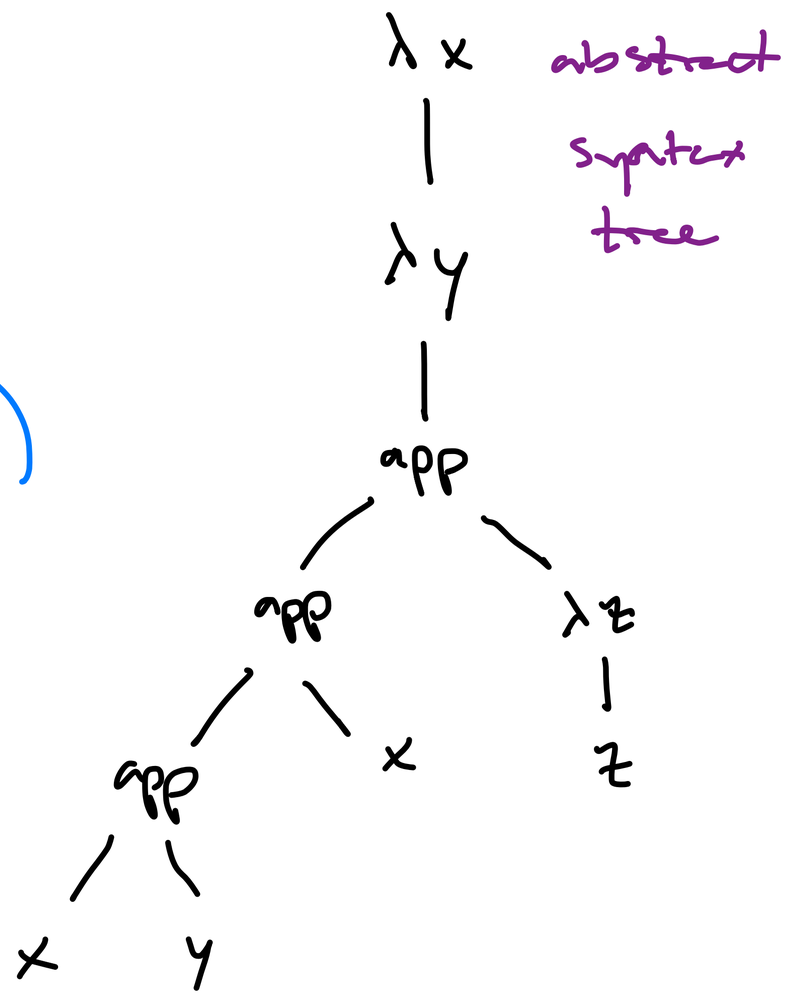
var

fun x -> fun y -> x y x (fun z -> z)

$$\lambda x.(\lambda y.((((x\ y)\ x)(\lambda z.z))))$$

AST

abstract syntax tree



Warning: Get used to λ

$$\lambda x . e \quad \equiv \quad \texttt{fun } x \texttt{ -> } e$$

We will use these syntaxes interchangeably starting now

These are the same thing, get used to it

Small-Step Semantics

Full (Nondeterministic) Small-step

$$\frac{e_1 \longrightarrow e_1'}{e_1\, e_2 \longrightarrow e_1'\, e_2}\ (\text{appLeft})$$

$$\frac{e_2 \longrightarrow e_2'}{e_1\, e_2 \longrightarrow e_1\, e_2'}\ (\text{appRight})$$

$$\frac{e \longrightarrow e'}{\lambda x.e \longrightarrow \lambda x.e'}\ (\text{fun})$$

$(\text{fun } x \to e_1)\ e_2$

redex

$$(\lambda x.e_1)\, e_2 \longrightarrow [e_2/x]\, e_1 \quad (\beta)$$

binder

sub. e_2 for x in e_1

$$(\ldots((\lambda x.e_1)\, e_2)\ldots) \longrightarrow (\ldots([e_2/x]\, e_1)\cdots)$$

Example

$$(\lambda f . \lambda x . fx)(\lambda y . y)$$

higher order app

identity

$$\frac{}{(\lambda f.\lambda x.fx)(\lambda y.y) \longrightarrow \lambda x.(\lambda y.y)x}\ (\beta)$$

$$[\lambda y.y / f](\lambda x.fx)$$

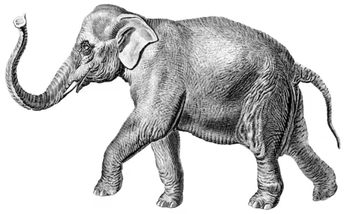
$$[x/y]y$$

$$\frac{\dfrac{}{(\lambda y.y)x \longrightarrow x}\ (\beta)}{\lambda x.(\lambda y.y)x \longrightarrow \lambda x.x}\ (fun)$$

$$(\lambda f.\lambda x.fx)(\lambda y.y) \longrightarrow^{*} \lambda x.x$$

Substitution

fun (fun y->y) -> fun y->y



$[\lambda y.y \,/\, x](\lambda x.x) = \lambda(\lambda y.y).(\lambda y.y)$

no sub.
in binder var.

identity

$[\lambda y.y \,/\, x](\lambda x.x) = \lambda x.\lambda y.y$

constant

α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x\,.\,\lambda y\,.\,x =_\alpha \lambda v\,.\,\lambda w\,.\,v$$

λ-calculus

α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x\,.\,\lambda y\,.\,x =_\alpha \lambda v\,.\,\lambda w\,.\,v$$

λ-calculus

The **principle of name irrelevance** says that any two programs that are the same up to "renaming of variables" should behave exactly the same way (they are **α-equivalent**)

α-Equivalence

```
let x = 2 in x + 1
```

$=_\alpha$

```
let z = 2 in z + 1
```

OCaml

$$\lambda x\,.\,\lambda y\,.\,x =_\alpha \lambda v\,.\,\lambda w\,.\,v$$

λ-calculus

The **principle of name irrelevance** says that any two programs that are the same up to "renaming of variables" should behave exactly the same way (they are **α-equivalent**)

We say that a variable x is **bound** is an expression if it appears in the expression as $(\ldots\lambda x\,.\,e\ldots)$

α-Equivalence

```
let x = 2 in x + 1
      =α
let z = 2 in z + 1
```

OCaml

$$\lambda x\,.\,\lambda y\,.\,x =_\alpha \lambda v\,.\,\lambda w\,.\,v$$

λ-calculus

The **principle of name irrelevance** says that any two programs that are the same up to "renaming of variables" should behave exactly the same way (they are **α-equivalent**)

We say that a variable x is **bound** is an expression if it appears in the expression as $(\ldots\lambda x\,.\,e\ldots)$

Substitution should preserve this

Preserving α-Equivalence

$$e_1 =_\alpha e_2 \Rightarrow f(e_1) =_\alpha f(e_2)$$

$$\lambda x.x =_\alpha \lambda z.z$$

but

$$[\lambda y.y / x](\lambda x.x) = \lambda x.\lambda y.y \neq_\alpha \lambda z.z$$

$$[\lambda y.y / x](\lambda z.z)$$

Definition (First Attempt)

Definition (First Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases} \quad (1)$$

$$[v/y](\lambda x\,.\,e) = \lambda x\,.\,[v/y]e \quad (2)$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2) \quad (3)$$

Definition (First Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases} \quad (1)$$

$$[v/y](\lambda x\,.\,e) = \lambda x\,.\,[v/y]e \quad (2)$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2) \quad (3)$$

1. Replace every y with v, leave other variables

Definition (First Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases} \quad (1)$$

$$[v/y](\lambda x\,.\,e) = \lambda x\,.\,[v/y]e \quad (2)$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2) \quad (3)$$

1. Replace every y with v, leave other variables
2. Replace y with v in the body of a function

Definition (First Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases} \quad (1)$$

$$[v/y](\lambda x\,.\,e) = \lambda x\,.\,[v/y]e \quad (2)$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2) \quad (3)$$

1. Replace every y with v, leave other variables
2. Replace y with v in the body of a function
3. Replace y with v in both subexpressions of an application

(This is an example of an *inductive definition*)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$
$$[v/y](\lambda x\,.\,e) = \lambda x\,.\,[v/y]e$$
$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Problem Case I

$$[y/x](\lambda x\,.\,x)$$

We shouldn't be allowed to substitute x if it's the argument of a function

This may *change the behavior* of a function

Definition (Second Attempt)

$$[e/x]\,y = \begin{cases} e & y = x \\ y & y \neq x \end{cases}$$

$$[\lambda y.y/x](\lambda x.x) =$$

$$\lambda x.x =_\alpha$$

$$\lambda z.z$$

$$[e/x](e_1\,e_2) = ([e/x]\,e_1)\,([e/x]\,e_2)$$

$$[\lambda y.y/x](\lambda z.z)$$

$$[e/x](\lambda y.e') = \begin{cases} \lambda y.e' & x = y \\ \lambda y.[e/x]e' & x \neq y \end{cases}$$

Definition (Second Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

We can handle the problem case directly in our definition. *Check the bound variable before we substitute in the body of a function*

Is there still a problem?

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Problem Case II

constant

$$[y/x](\lambda y\,.\,x) =$$

$$\lambda y.\,[y/x]\,x =$$

$$\lambda y.\,y$$

identity

We're not replacing a bound variable, but we *are* substituting an expression that has variables which *became* bound

The variable y is said to be **captured** in this (incorrect) substitution

Free Variables

Free and Bound Variables

$$FV(x) = \{x\} \tag{1}$$

$$FV(\lambda x\,.\,e) = FV(e) \backslash \{x\} \tag{2}$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \tag{3}$$

Free and Bound Variables

$$FV(x) = \{x\} \tag{1}$$

$$FV(\lambda x\,.\,e) = FV(e) \backslash \{x\} \tag{2}$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \tag{3}$$

Def. A variable x is **free** in e if it does not appear **bound** by a λ. Formally:

Free and Bound Variables

$$FV(x) = \{x\} \quad (1)$$

$$FV(\lambda x\,.\,e) = FV(e) \backslash \{x\} \quad (2)$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \quad (3)$$

Def. A variable x is **free** in e if it does not appear **bound** by a λ. Formally:

1. x is free in x

Free and Bound Variables

$$FV(x) = \{x\} \quad (1)$$

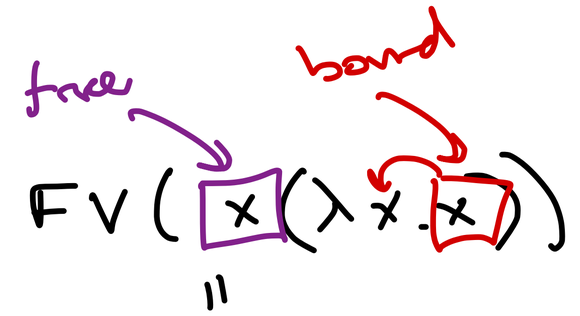
$$FV(\lambda x\,.\,e) = FV(e) \backslash \{x\} \quad (2)$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \quad (3)$$

Def. A variable x is **free** in e if it does not appear **bound** by a λ. Formally:

1. x is free in x
2. x is free in $\lambda y.e$ if it is free in e and $x \neq y$

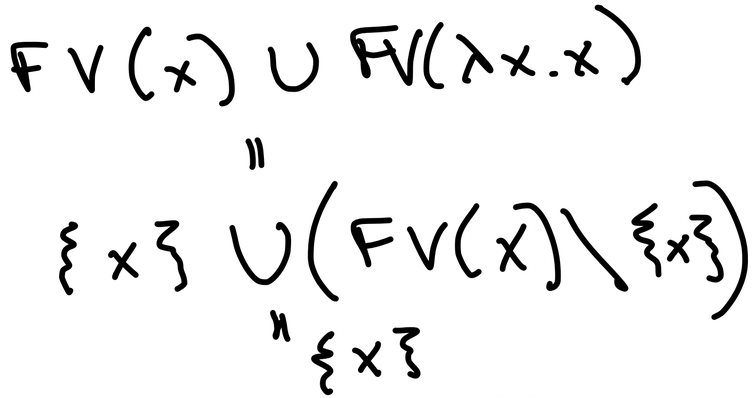
Free and Bound Variables



$$FV(x) = \{x\} \quad (1)$$

$$FV(\lambda x\,.\,e) = FV(e) \backslash \{x\} \quad (2)$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \quad (3)$$



Def. A variable x is **free** in e if it does not appear **bound** by a λ. Formally:

1. x is free in x
2. x is free in $\lambda y.e$ if it is free in e and $x \neq y$
3. x is free in $e_1 e_2$ if x is free in e_1 or e_2

Free and Bound Variables

$$FV(x) = \{x\} \quad (1)$$

$$FV(\lambda x\,.\,e) = FV(e) \backslash \{x\} \quad (2)$$

$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2) \quad (3)$$

Def. A variable x is **free** in e if it does not appear **bound** by a λ. Formally:

1. x is free in x
2. x is free in $\lambda y.e$ if it is free in e and $x \neq y$
3. x is free in $e_1 e_2$ if x is free in e_1 or e_2

Def. A variable x is **free** in e if $x \in FV(e)$ as above

Definition (Third Attempt)

Definition (Third Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v) \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Definition (Third Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v) \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Since we're interested in α-equivalence, we can first *replace* the bound variable and *substitute* it in the body of the function. This is called **α-renaming**

Definition (Third Attempt)

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v) \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Since we're interested in α-equivalence, we can first *replace* the bound variable and *substitute* it in the body of the function. This is called **α-renaming**

Is there still a problem?

Problem Case III

$$FV(x) = \{x\}$$
$$FV(\lambda x\,.\,e) = FV(e) \backslash \{x\}$$
$$FV(e_1 e_2) = FV(e_1) \cup FV(e_2)$$

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[w/z][z/x]e & x \in FV(v) \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

$$[x/y](\lambda x\,.\,xyz)$$

This isn't exactly a problem, but *we have to be careful about which variable to replace the bound variable* x *with*

If we choose z, then we capture a *different* variable!

Capture-Avoiding Substitution

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Capture-Avoiding Substitution

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Finally a definition, that works. Sort of...

Capture-Avoiding Substitution

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Finally a definition, that works. Sort of...

The only problem with this definition is that it now poses an implementation issue. **How do we come up with z?**

Capture-Avoiding Substitution

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda z\,.\,[v/y][z/x]e & x \in FV(v), z \texttt{ is fresh} \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Finally a definition, that works. Sort of...

The only problem with this definition is that it now poses an implementation issue. **How do we come up with z?**

In mathematics, we can say it's **always possible** to come up with a variable z, but when we're implementing a programming language, we need an *actual* procedure

Evaluation under binders

$$\frac{e_1 \longrightarrow e_2}{e_1 e \longrightarrow e_2 e}\ \textsc{appLeft} \qquad \frac{e_1 \longrightarrow e_2}{e e_1 \longrightarrow e e_2}\ \textsc{appRight}$$

$$\frac{}{(\lambda x.e_1)e_2 \longrightarrow [e_2/x]e_1}\ \textsc{beta}$$

$$v ::= \lambda x.e$$

weak head normal form

We generally don't need to evaluate under binders in practice

This means dropping the (fun) rule and updating our notion of values

Well-Scopedness and Closedness

open

$$\lambda x \,.\, y$$

closed

$$\lambda x \,.\, \lambda y \,.\, y$$

Def. *(informal)* An expression e is **well-scoped** if every variable in e is "in scope" (more on that on Thursday)

Def. An expression e is **closed** if it has no free variables

Every closed term is well-scoped

Non-Capture-Avoiding Substitution

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Non-Capture-Avoiding Substitution

$$[v/y]x = \begin{cases} v & x = y \\ x & \texttt{else} \end{cases}$$

$$[v/y](\lambda x\,.\,e) = \begin{cases} \lambda x\,.\,e & x = y \\ \lambda x\,.\,[v/y]e & \texttt{else} \end{cases}$$

$$[v/y](e_1 e_2) = ([v/y]e_1)([v/y]e_2)$$

Thm. If e is closed (well-scoped), then this definition equivalent to capture avoiding substitution

Evaluation Strategies

Another (Deterministic) Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad (2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

Another (Deterministic) Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

1. We can reduce the LHS of an application

Another (Deterministic) Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1) e_2 \longrightarrow (\lambda x\,.\,e_1) e_2'}$$

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

1. We can reduce the LHS of an application
2. We can reduce the RHS of an application *if the LHS is already a function*

Another (Deterministic) Small-Step Semantics

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x\,.\,e_1)e_2 \longrightarrow (\lambda x\,.\,e_1)e_2'}$$

$$(3)\ \frac{}{(\lambda x\,.\,e)(\lambda y\,.\,e') \longrightarrow [(\lambda y\,.\,e')/x]e}$$

1. We can reduce the LHS of an application
2. We can reduce the RHS of an application *if the LHS is already a function*
3. We can apply a function to value via β

Example

$$(\lambda f \,.\, \lambda x \,.\, fx)(\lambda y \,.\, y)$$

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad (2)\ \frac{e_2 \longrightarrow e_2'}{(\lambda x \,.\, e_1) e_2 \longrightarrow (\lambda x \,.\, e_1) e_2'}$$

$$(3)\ \frac{}{(\lambda x \,.\, e)(\lambda y \,.\, e') \longrightarrow [(\lambda y \,.\, e')/x]e}$$

Small-Step Semantics (Another Form)

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

Small-Step Semantics (Another Form)

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

1. We can reduce the LHS of an application

Small-Step Semantics (Another Form)

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}$$

$$(2)\ \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

1. We can reduce the LHS of an application
2. We can apply a function to *any* expression via substitution

Example

$$(\lambda x\,.\,y)((\lambda z\,.\,z)(\lambda w\,.\,w))$$

$$(1)\ \frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad (2)\ \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

Stuck Terms

<val> ::= λ<var>.<expr>

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

$$(\lambda x\,.\,yx)(\lambda x\,.\,x)$$

Based on our operational semantics, it's not possible for the above expression to reduce to a value

Non-Termination

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2} \qquad \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}$$

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

And unlike with arithmetic, it's now possible to define expressions which **do not terminate.** These expression do not have values, but also don't get stuck

Big-Step Semantics

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

Big-Step Semantics

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

1. A function evaluates to a function value (itself)

Big-Step Semantics

$$(1)\ \frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$(2)\ \frac{e_1 \Downarrow \lambda x\,.\,e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

1. A function evaluates to a function value (itself)

2. If e_1 evaluates to the function $\lambda x\,.\,e$ and e_2 evaluates to the value v_2 and e with v_2 substituted for x evaluates to v, then the application $e_1 e_2$ evaluates to v

Big-Step Semantics (Another Form)

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e \qquad [e_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}$$

These are the same rules as before except we're not required to evaluate e_2 first

Practice Problem

$$(\lambda x\,.\,\lambda y\,.\,y)((\lambda z\,.\,z)(\lambda q\,.\,q)) \Downarrow \lambda y\,.\,y$$

Give a derivation of the above judgment in both versions of the big-step semantics

$$\frac{\dfrac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \dfrac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \dfrac{}{[v_2/x]e \Downarrow v}}{e_1 e_2 \Downarrow v}$$

$$\frac{\dfrac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \dfrac{}{[e_2/x]e \Downarrow v}}{e_1 e_2 \Downarrow v}$$

Answer

$$(\lambda x\,.\,\lambda y\,.\,y)((\lambda z\,.\,z)(\lambda q\,.\,q)) \Downarrow \lambda y\,.\,y$$

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBN)}$$

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBN)}$$

The two versions of semantics we've given correspond to **call-by-value** (CBV) and **call-by-name** (CBN)**.** These are **evaluation strategies** for functional PLs

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBN)}$$

The two versions of semantics we've given correspond to **call-by-value** (CBV) and **call-by-name** (CBN)**.** These are **evaluation strategies** for functional PLs

CBV: evaluate the argument of a function *before* substituting it in the function

Call-by-Value vs. Call-by-Name

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBV)}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}\,\text{(CBN)}$$

The two versions of semantics we've given correspond to **call-by-value** (CBV) and **call-by-name** (CBN)**.** These are **evaluation strategies** for functional PLs

CBV: evaluate the argument of a function *before* substituting it in the function

CBN: substitute the expression *directly* into the function

Benefits of CBV

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,x + x + x + x)e$$

Benefits of CBV

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,x + x + x + x)e$$

If we compute the value of an argument before substituting it into the expression, we only have to compute the expression *once*

Benefits of CBV

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,x + x + x + x)e$$

If we compute the value of an argument before substituting it into the expression, we only have to compute the expression *once*

This is good if the variable appears several times in the body of our function

Benefits of CBV

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,x + x + x + x)e$$

If we compute the value of an argument before substituting it into the expression, we only have to compute the expression *once*

This is good if the variable appears several times in the body of our function

This is also called **eager**, or **applicative**, or **strict** evaluation (and is what OCaml does)

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1 e_2$$

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1e_2$$

If a variables doesn't appear in our function, then the argument is *not evaluated at all*

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e}$$

$$\frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1e_2$$

If a variables doesn't appear in our function, then the argument is *not evaluated at all*

If an argument is only seldomly used, it will only be computed when it is used (e.g, if its computed in a branch of an if-expression that is almost never reached)

Benefits of CBN

$$\frac{}{\lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{e_1 \Downarrow \lambda x\,.\,e_1' \qquad [e_2/x]e_1' \Downarrow v}{e_1 e_2 \Downarrow v}$$

$$(\lambda x\,.\,\lambda y\,.\,x)e_1e_2$$

If a variables doesn't appear in our function, then the argument is *not evaluated at all*

If an argument is only seldomly used, it will only be computed when it is used (e.g, if its computed in a branch of an if-expression that is almost never reached)

Aside. It's possible to simulate CBN in CBV. Think about it for a bit, ask me after if you're interested

Aside: Side Effects

```
let f x = x + x in
let y =
  let _ = print_int 2 in
  2
in f y
```

Aside: Side Effects

```
let f x = x + x in
let y =
  let _ = print_int 2 in
  2
in f y
```

What does this program print?

Aside: Side Effects

```
let f x = x + x in
let y =
  let _ = print_int 2 in
  2
in f y
```

What does this program print?

It depends on if we're using CBN or CBV evaluation

Aside: Side Effects

```
let f x = x + x in
let y =
  let _ = print_int 2 in
  2
in f y
```

What does this program print?

It depends on if we're using CBN or CBV evaluation

Def. *(informal)* A **side effect** refers to something that happens during the evaluation of a program that is not a part of the formal semantics, e.g., printing to the console

Aside: Side Effects

```
let f x = x + x in
let y =
  let _ = print_int 2 in
  2
in f y
```

What does this program print?

It depends on if we're using CBN or CBV evaluation

Def. *(informal)* A **side effect** refers to something that happens during the evaluation of a program that is not a part of the formal semantics, e.g., printing to the console

Different evaluation strategies yield different side-effectful behavior

Aside: Evaluation Strategies

Aside: Evaluation Strategies

There are *a lot* more evaluation strategies, all of which optimize *something*

Aside: Evaluation Strategies

There are *a lot* more evaluation strategies, all of which optimize *something*

Haskell uses **lazy** evaluation also called **call-by-need**

Aside: Evaluation Strategies

There are *a lot* more evaluation strategies, all of which optimize *something*

Haskell uses **lazy** evaluation also called **call-by-need**

In languages with pointers we also often have the option to use **call-by-reference** evaluation or **call-by-sharing**

demo

(lambda calculus)

Summary

The **lambda calculus** is a simple function programming language useful for learning basic functional concepts

Different **evaluation strategies** have different benefits (and may yield different programming behaviors)

Substitution is a bit tricky to define correctly (!) but any definition must preserve α–equivalence